Functions as arguments and results

A higher-order function takes a function as an argument or returns a function as a result. Haskell functions are first-class values: they can be passed to other functions and stored in data structures.

map applies a function to every element of a list, preserves order, and preserves the length of a finite list.

```
map :: (a -> b) -> [a] -> [b]
map (*2) [1,2,3] == [2,4,6]
```

filter retains exactly the elements satisfying a predicate, preserving their order. A predicate returns Bool.

```
filter even [1,2,3,4] == [2,4]
```

A lambda is an anonymous function. The expression \x -> x + 1 adds one to its argument. map (\x -> x + 1) [4,5] returns [5,6].

Function composition connects outputs to inputs: (f . g) x = f (g x). For f = (*2) and g = (+1), (f . g) 3 is 8. Partial application supplies fewer arguments than a function expects, producing another function; add 3 is a function waiting for its second argument.

Original educational material / CC BY 4.0 / Bilal Amin

Folds and evaluation order

A fold replaces list constructors with a combining operation and a starting value.

```
foldr f z [] = z
foldr f z (x:xs) = f x (foldr f z xs)
```

For a finite list [1,2,3], foldr f z expands to f 1 (f 2 (f 3 z)). A left fold associates the accumulator to the left:

```
foldl f z [1,2,3] = f (f (f z 1) 2) 3
```

Non-associative operations expose the difference: foldr (-) 0 [1,2,3] equals 2, while foldl (-) 0 [1,2,3] equals -6. For addition on finite lists, both produce the same mathematical sum.

Use foldr when the combining function can produce output without evaluating the entire recursive result, such as constructing a lazy output list. With a suitable short-circuiting function, foldr can work on some infinite lists; this does not mean every foldr on an infinite list terminates.

For a strict accumulated numeric result on a large finite list, prefer Data.List.foldl' to avoid a chain of unevaluated accumulator expressions. Ordinary foldl is not automatically strict in Haskell.

```
sumWithFold = foldr (+) 0
copyList = foldr (:) []
```

Original educational material / CC BY 4.0 / Bilal Amin